1 - Ivo sale a correr dando vueltas alrededor de un parque. La vuelta completa tiene una longitud de T cuadras, de las cuales exactamente K son cuadras en subida. Ivo inicia su recorrido en la cuadra de salida al parque y comienza a correr dando vueltas hasta completar un total de N cuadras. Notar que N no tiene por qué ser necesariamente múltiplo de T, por lo cual Ivo no necesariamente corre una cantidad exacta de vueltas completas. Dependiendo de dónde estén ubicadas exactamente las K cuadras en subida y de dónde comienza Ivo a correr. Tu tarea es escribir un programa que, dados T, K y N, determine la cantidad de cuadras en subida que tiene que correr Ivo.

Especificación del programa:

- Parámetros:
    1. T: Entero que indica la longitud total de la vuelta (en cuadras).
    2. K: Entero que indica la cantidad de cuadras en subida en una vuelta completa.
    3. N: Entero que indica la cantidad total de cuadras que corre Ivo.
- Valor de retorno:
    1. El número que indica la cantidad de subidas que recorre Ivo.

Ejemplo:

- Entrada: T = 3, K = 1, N = 10
- Salida: 3

Para avanzar al siguiente problema, ingresa el resultado mínimo que arroja tu programa para la siguiente entrada de prueba oculta:

- Entrada de evaluación: T = 1367, K = 486, N = 85472
- Formato de respuesta esperado en el sistema: Un número

2 - En una ciudad existe un servicio de bicicletas de alquiler con N estaciones. Cada estación tiene espacio para B bicicletas (B es par), y todas las mañanas debe haber en cada estación la mitad de los espacios ocupados, de forma tal que haya B / 2 bicicletas para retirar y B / 2 espacios para devolver bicicletas. En consecuencia, la cantidad total de bicicletas disponible en la ciudad es T = (N * B) / 2. Durante el día, los ciudadanos alquilan las bicicletas, y cuando termina el día las dejan en diferentes estaciones, de acuerdo al patrón de uso que hayan tenido de las mismas.

A la noche, los empleados del servicio de alquiler deben redistribuir las bicicletas, de forma tal que a la mañana cada estación tenga nuevamente B / 2 bicicletas.

El problema a resolver es el siguiente: sabiendo la cantidad de bicicletas en cada estación al final del día, calcular la mínima cantidad de bicicletas que se deben mover de forma tal que al otro día haya B / 2 bicicletas en cada estación.

Especificación del programa:

- Parámetros:
    1. c: Arreglo de N enteros que describen las estaciones. La i-ésima estación tiene c[i] bicicletas a la noche.
- Valor de retorno:
    1. El número entero que indica la mínima cantidad de bicicletas que es necesario mover durante la redistribución nocturna.

Ejemplos:

- Ejemplo 1:
    - Entrada: N = 4, c = [2, 4, 0, 2] *(asumiendo B = 4)*
    - Salida: 2

Para avanzar al siguiente problema, ingresa el resultado que arroja tu programa para la siguiente entrada de prueba oculta:

- Entrada de evaluación: N = 20, B = 20, c = [15, 4, 18, 2, 10, 12, 8, 14, 6, 11, 9, 16, 3, 17, 5, 13, 7, 10, 12, 8]
- Formato de respuesta esperado en el sistema: Un número

3 - Bernardo el vendedor tiene que dar un vuelto. Esto requiere rápidos cálculos mentales para dar el vuelto con la mínima cantidad de monedas posible. Más importante aún, esto requiere gran destreza manual para manipular las monedas, y ese no es el fuerte de Bernardo. Existe una lista de N denominaciones de monedas, de a1 centavos, a2 centavos, ..., an-1 centavos y an centavos.Bernardo posee un suministro ilimitado de monedas de cada una de estas denominaciones. Bernardo utilizó lo que aprendió estudiando robótica para programar y construir un robot automático de entrega de vuelto que lo ayude en su negocio. Programó al robot utilizando un algoritmo denominado goloso, avaro o voraz (se les llama greedy en inglés). En el algoritmo, se consideran las denominaciones disponibles ai, y el monto restante de V centavos que falta entregar. Se elige en cada paso la más grande posible ak tal que ak <= V, es decir sin pasarse del vuelto restante por entregar V. Se agrega una moneda de esa denominación ak a la solución, y se resta su valor al vuelto restante V, es decir se realiza la asignación V <- V - ak. El algoritmo continúa hasta que sea V = 0.

Tu tarea consiste en ayudar a Bernardo a programar un programa que le indique si su programa es correcto, y si no lo es,le indique el menor posible de V para el cual su programa no utiliza la mínima cantidad posible de monedas.

Especificación del programa:

- Parámetros:
    1. a: Arreglo de N enteros con las denominaciones de monedas ai.
- Valor de retorno:
    1. Un único entero positivo que indique el mínimo valor de V tal que el algoritmo de Bernardo falla (es decir,no entrega la mínima cantidad de monedas posible para dar un vuelto V). Si para las denominaciones indicadas el algoritmo es correcto, se debe retornar -1.

Ejemplos:

- Ejemplo 1:
    - Entrada: N = 4, a = [1, 5, 11, 20]
    - Salida: 15 *(Ya que V = 15 es el primer fallo del algoritmo de Bernardo: usará cinco monedas formando 15 = 11 + 1 + 1 + 1 + 1, pero se puede hacer 15 = 5 + 5 + 5 usando solamente tres monedas).*
- Ejemplo 2:
    - Entrada: N = 4, a = [1, 5, 15, 30]
    - Salida: -1

Para avanzar al siguiente problema, ingresa el resultado que arroja tu programa para la siguiente entrada de prueba oculta:

- Entrada de evaluación: N = 12, a = [1, 6, 14, 29, 58, 115, 230, 460, 920, 1840, 3680, 7360]
- Formato de respuesta esperado en el sistema: Un número

4 - En este desafío nos adentraremos en el mundo de los autómatas celulares y la evolución artificial. Imagina un arreglo unidimensional de N celdas, donde cada celda puede tener un estado de 1 (activa) o 0 (inactiva).

El sistema evoluciona a lo largo de E ciclos de tiempo. Las reglas del proceso son las siguientes:

1. Inicialización algorítmica: El arreglo inicial de tamaño N se genera mediante una condición matemática. Para cada posición i (desde 0 hasta N - 1), el valor inicial es 1 si la condición (i * 7 + 3) % 5 == 0 se cumple, y 0 en caso contrario.
2. Ciclos de evolución: Se deben simular E ciclos consecutivos. En cada ciclo, cada celda se actualiza basándose en sus vecinos inmediatos (su valor anterior, el de su izquierda y el de su derecha, utilizando condiciones de frontera circulares: el vecino a la izquierda de la celda 0 es la celda N - 1, y el vecino a la derecha de la celda N - 1 es la celda 0).
    - La regla de transición para el siguiente estado de la celda i es: si la suma de los valores de la celda i y sus dos vecinos (izquierda y derecha) en el ciclo anterior es un número **par**, la celda pasa a ser 1; si la suma es **impar**, pasa a ser 0.

Tu tarea es escribir un programa que, dados los valores de N y E, ejecute la carga inicial y los E ciclos de evolución descritos, y finalmente retorne la suma total de todos los valores del arreglo resultante.

Especificación del programa:

- Parámetros:
    1. N: Entero que indica la cantidad de celdas (tamaño del arreglo).
    2. E: Entero que indica la cantidad de ciclos de evolución a simular.
- Valor de retorno:
    1. Un número entero que representa la suma total de los valores de todas las celdas del arreglo tras completar los E ciclos.

Ejemplos:

- Ejemplo 1:
    - Entrada: N = 10, E = 3
    - Salida: 4

Para avanzar al siguiente problema, ingresa el resultado que arroja tu programa para la siguiente entrada de prueba oculta:

- Entrada de evaluación: N = 300, E = 72

- Formato de respuesta esperado en el sistema: Un número